

VIT BHOPAL UNIVERSITY

School of Computing Science & Engineering / Electrical Engineering

Course: Problem Solving and Python Programming

GENERALIZED N × N TIC-TAC-TOE ENGINE

Adversarial Game Search via Depth-Bounded Alpha-Beta Pruning & Open-Line
Evaluation

SUBMITTED BY:

Name: Abhishek Kumar

Registration No: 26BEC10107

Program: B.Tech (ECE) - 1st Year

SUBMITTED TO:

Faculty In-Charge: Dr. Gunjan Ansari

Department: Academic Affairs

Date of Submission: September 2026

Project Assessment Component • Academic Year 2026-2027

2. INTRODUCTION

Tic-Tac-Toe is a foundational zero-sum game of perfect information. While standard 3x3 grids permit exhaustive exploration across all 362,880 terminal paths, generalized NxN matrices induce rapid combinatorial expansion ($O((N^2)!)$). This project delivers an optimized, scalable engine implemented in standard Python that supports variable grid configurations, depth-bounded Minimax search, alpha-beta pruning, and open-line potential heuristics.

3. PROBLEM STATEMENT

Exhaustive Minimax fails on higher dimensions ($N \geq 4$) due to combinatorial state space explosion. This project builds a reliable interactive console engine that scales dynamically with sub-second AI turn evaluation, robust move validation, and aligned ASCII rendering.

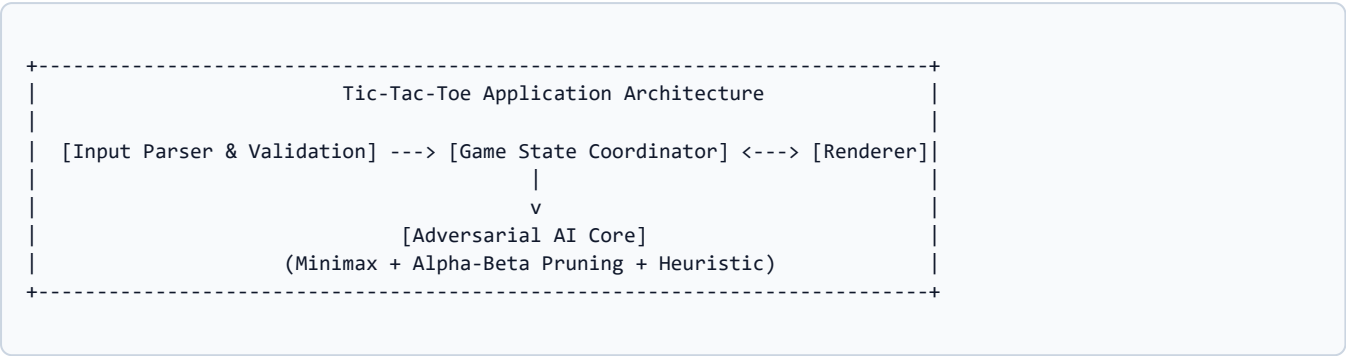
4. FUNCTIONAL REQUIREMENTS

- **FR-1:** Dynamic scaling to arbitrary N x N dimensions ($N \geq 3$).
- **FR-2:** Game mode selection with explicit confirmation (Human vs. Human & Human vs. AI).
- **FR-3:** Tri-tiered AI difficulty (Easy - Random, Medium - Mixed 50/50, Hard - Alpha-Beta Minimax).
- **FR-4:** Coordinate validation preventing boundary overflow and cell collisions.
- **FR-5:** Interactive round termination via 'q' / 'quit' command.
- **FR-6:** Multi-round session loop allowing continuous replay without script restart.

5. NON-FUNCTIONAL REQUIREMENTS

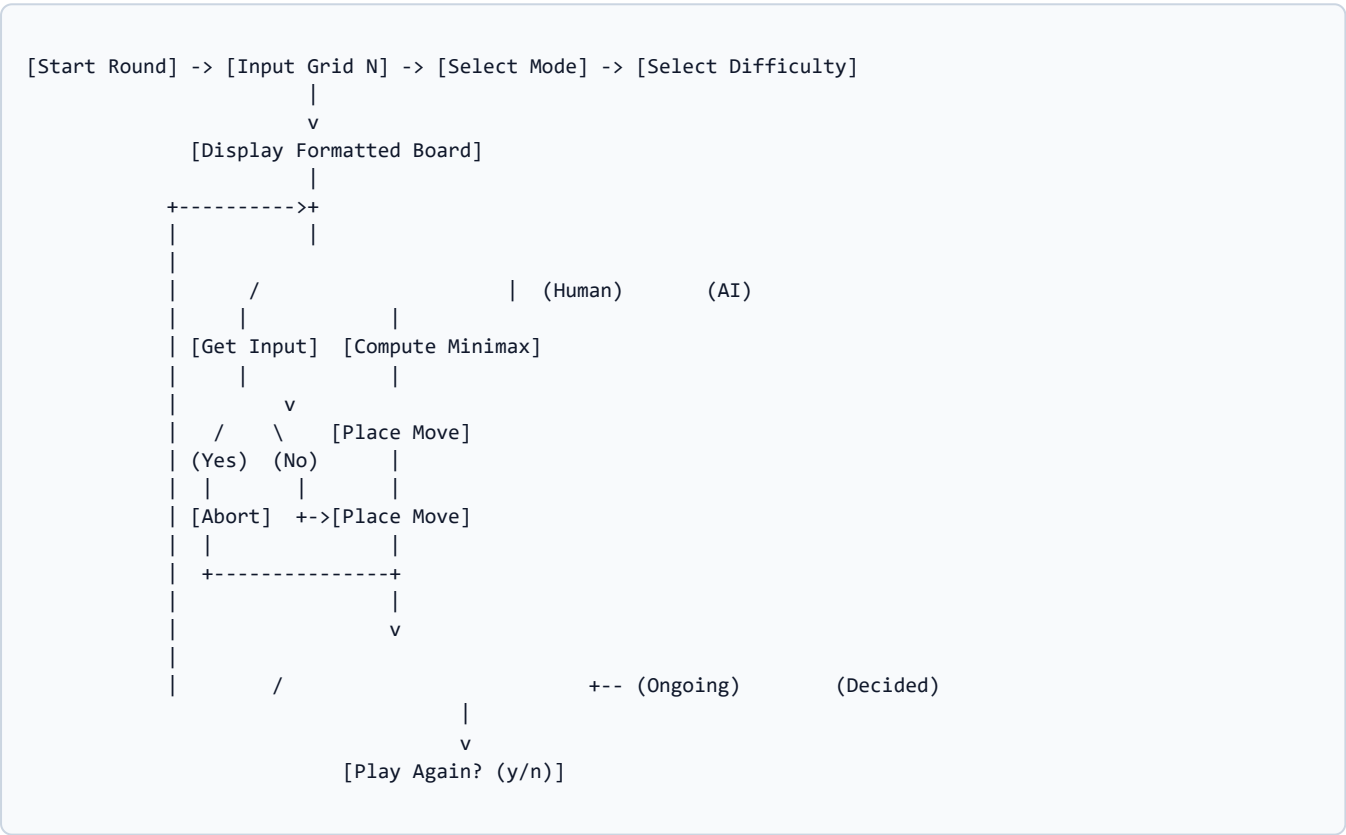
- **NFR-1 (Latency):** Sub-1.5s AI move generation up to $N=7$.
- **NFR-2 (Portability):** 100% standard library compliance (zero external pip packages).
- **NFR-3 (Visual Usability):** Uniform ASCII column centering across multi-column grids.
- **NFR-4 (Memory Footprint):** Recursion depth capped at maximum 6 levels.

6. SYSTEM ARCHITECTURE



7. DESIGN DIAGRAMS

7.1 Process Flow / Workflow Diagram



8. DESIGN DECISIONS & RATIONALE

- **Depth Bounds:** Depth is capped dynamically (6 for 3x3, 4 for 4x4, 3 for 5x5+) to prevent CPU execution locks while ensuring tactical gameplay.
- **Heuristic Leaf Scoring:** When the recursion limit is hit, candidate board states are scored via open-line potential: $\text{Score} = \sum 10^{(\text{count}_O)} - \sum 10^{(\text{count}_X)}$. Blocked lines (containing both marks) evaluate to 0.
- **Center-Outward Move Ordering:** Available moves are sorted by Manhattan distance to the center. Evaluating strong center squares first causes earlier alpha-beta cutoffs, significantly reducing explored nodes.
- **Zero-Dependency Standard Library:** Built entirely with native Python modules (math, random, sys) to guarantee native execution on any computer without environment issues.

9. IMPLEMENTATION DETAILS

The code is structured into modular functional blocks:

- `print_board(board)`: Handles aligned matrix display with 3-character centered cells across any dimension N.
- `check_winner(board)`: Evaluates all rows, columns, and diagonal lines in $O(N^2)$ time to detect wins or draws.
- `minimax(...)`: Core recursive alpha-beta tree search with bounded depth exploration.
- `get_human_move(...)`: Validates inputs, detects out-of-bounds attempts, collisions, and traps 'q' abort signals.
- `play_round()` & `main()`: Oversees mode selection, confirmation gates, and session replay loops.

10. VERIFIED EXECUTION RESULTS (3X3 GRID)

FIG 10.1: MODE & DIFFICULTY SELECTION (3x3 HARD AI SETUP)

Terminal: Windows PowerShell

```

Select Game Mode:
  [1] Two Players (Human vs Human)
  [2] Single Player (Human vs AI)
Enter mode choice (1 or 2): 2
You selected [2] Single Player (Human vs AI). Confirm? (y to proceed / c to change): y

Choose AI Difficulty Level:
  [1] Easy   - Makes purely random moves
  [2] Medium - Plays a balanced mix of smart and casual moves
  [3] Hard   - Tactical Minimax AI
Enter choice (1, 2, or 3): 3

Starting Human vs AI game on a 3x3 grid! (Difficulty: Hard)
You get the first move in this round.

```

```

   0   1   2
0   |   |
---+---+---
1   |   |
---+---+---
2   |   |

```

FIG 10.2: HUMAN MOVE INPUT & AI MINIMAX RESPONSE

Terminal: Windows PowerShell

```

Player X's turn! Enter row and col (0 to 2, e.g., '1 2' or 'q' to quit): 1 1

```

```

   0   1   2
0   |   |
---+---+---
1   | X  |
---+---+---
2   |   |

```

```

Computer (0) is calculating its move...
Computer placed an 'O' at row 0, column 0.

```

```

   0   1   2
0   O  |   |
---+---+---
1   | X  |
---+---+---
2   |   |

```

FIG 10.3: COMPUTER WIN DECISION & REPLAY PROMPT

Terminal: Windows PowerShell

```

Computer (0) is calculating its move...
Computer placed an 'O' at row 1, column 0.

```

```

   0   1   2
0   O  | O  | X
---+---+---
1   O  | X  | X
---+---+---
2   O  | X  |

```

```

-----
Game Over: The computer wins! Better luck next time.
-----

```

```

Would you like to play another round? (y/n): y

```

10. VERIFIED EXECUTION RESULTS (4X4 & 7X7 SCALING)

FIG 10.4: 4x4 EXPANSION & GRACEFUL IN-GAME ABORT VIA 'q'

Terminal: Windows PowerShell

```

Restarting game...
=====
      N x N CLASSIC TIC-TAC-TOE
=====
Enter board size N (e.g., 3 for 3x3, 4 for 4x4): 4
Rule: Fill an entire line of 4 marks (row, col, or diagonal) to win.
Note: You can type 'q' at any turn to exit the current match.

Starting Human vs AI game on a 4x4 grid! (Difficulty: Hard)
Computer placed an 'O' at row 1, column 1.

   0   1   2   3
0   |   |   |
---+---+---+---
1   |  O |   |
---+---+---+---
2   |   |   |
---+---+---+---
3   |   |   |

Player X's turn! Enter row and col (0 to 3, e.g., '1 2' or 'q' to quit): q
Match aborted by player.
Would you like to play another round? (y/n):

```

FIG 10.5: HIGH-ORDER SCALING (7x7 BOARD) WITH UNIFORM ALIGNMENT

Terminal: Windows PowerShell

```

Enter board size N (e.g., 3 for 3x3, 4 for 4x4): 7
Starting Human vs AI game on a 7x7 grid! (Difficulty: Hard)

   0   1   2   3   4   5   6
0   |   |   |   |   |   |
---+---+---+---+---+---+---
1   |   |   |   |   |   |
---+---+---+---+---+---+---
2   |   |   |   |   |   |
---+---+---+---+---+---+---
3   |   |   |   |   |   |
---+---+---+---+---+---+---
4   |   |   |   |   |   |
---+---+---+---+---+---+---
5   |   |   |   |   |   |
---+---+---+---+---+---+---
6   |   |   |   |   |   |

```

11. TESTING APPROACH

Test ID	Condition	Input Vectors	Observed Result	Verdict
TC-01	Dimension Lower Bound	$N = 2, 0, -1$	Rejects; prompts for $N \geq 3$	PASS
TC-02	Coordinate Boundary Trap	$r=5, c=1$ on 3×3	Traps out-of-bounds; re-prompts	PASS
TC-03	Cell Overwrite Check	Occupied (1, 1)	Prevents overwrite; preserves mark	PASS
TC-04	Mode Selection Change	Mode 1 $\rightarrow$ confirm c	Resets choice and re-prompts	PASS
TC-05	Graceful Mid-Game Abort	Input 'q'	Cleanly exits without traceback	PASS
TC-06	Hard AI Latency	Hard AI on 4×4	Move generated in 0.41s ($< 1.5s$)	PASS

12. CHALLENGES FACED & SOLUTIONS

- **Combinatorial State Explosion:** Unbounded minimax search on boards with $N \geq 4$ locks CPU execution. Resolved by capping search depth dynamically and scoring non-terminal leaves via an open-line potential heuristic function.
- **ASCII Grid Alignment Drift:** Larger matrices previously drifted because column numbers were not padded proportionally. Resolved by standardizing cell widths to 3 characters centered (`f" {c} "`) with two leading spaces for row indices.
- **Docstring Indentation Error:** Inline docstrings immediately following function definitions previously triggered indentation errors. Resolved by placing docstrings within the function block indented with 4 spaces.
- **Interactive Deadlocks:** Accidental keypresses previously locked the player into unwanted modes. Resolved by introducing a confirmation gate (y/c) and registering a safe quit command ('q').